

Nottsvale Floor Plan Build Guide and Bug Log

The interactive site floor plan code app: my architecture, the fully corrected code, and every bug I hit building it, with root cause and fix for each. Eleven bugs, five root causes. I wrote this up after the build so the mistakes are documented, not just fixed.

All the code in this guide compiles clean under `tsc --strict`. I checked it in a sandbox before writing it here, I didn't just transcribe it from memory.

What this app is

A React and TypeScript code app running inside the Power Platform. I pick a Nottsvale site, see its equipment plotted on a scale drawing, colour coded by how many maintenance requests are open against it, and click a machine for its history. An edit mode lets an admin click the drawing to place or move an asset, which writes back to Dataverse and persists.

Why a code app rather than canvas: Power Fx cannot render an interactive scale drawing with clickable hit regions. There's no low code route to it at all. This is the one part of the series where the argument isn't "canvas would be awkward," it's "canvas cannot do this."

Why it writes coordinates and not a lookup: Plan X and Plan Y are whole number columns. Writing a lookup from a code app is currently awkward. Microsoft's own guidance says a dedicated how to for it is still pending. I designed around that entirely rather than fighting it, which I think was the right call.

Architecture

```
src/
├─ types.ts           My own shapes, the app's contract
├─ App.tsx           Site selector, plan, detail/edit panel
├─ main.tsx         Entry point
├─ components/
│   └─ FloorPlan.tsx The drawing, markers, grid overlay
├─ hooks/
│   └─ useFloorPlanData.ts Fetch, derive, write, roll back
├─ utils/
│   ├── planMap.ts   Site to drawing, via SVG module imports
│   └─ grid.ts       cellRef() and gridCells(), display only
└─ generated/
    ├── models/
    └─ services/
```

The one hardcoded thing in the app is `planMap.ts`. The drawings are static assets, so something has to associate a site row with a file. Everything else comes from Dataverse. The production refinement would be a `Plan File` column on the Site table.

The grid is a display aid, not a data model. Assets store precise percentage coordinates. `cellRef()` derives a label like C4 for the panel, but nothing is ever stored as a grid reference. Four machines in the same cell would collapse to one reference if it were.

The full corrected code

Every file below reflects the state after all the bugs in my log were fixed. Import paths for the generated services are placeholders. See the note in `useFloorPlanData.ts` and the bug log entry on service naming before copying this.

`src/types.ts`

```

1 export type Site = { id: string; name: string };
2
3 export type Asset = {
4   id: string;
5   name: string;
6   tag: string;
7   siteId: string;
8   criticality?: string;
9   planX: number | null;
10  planY: number | null;
11  openCount: number;
12 };
13
14 export type MaintenanceRequest = {
15   id: string;
16   reference: string;
17   assetId: string;
18   status?: string;
19   priority?: string;
20   reportedOn?: string;
21   downtime?: number;
22 };

```

`planX` and `planY` are `number | null`. An asset can exist in the register before anyone has decided where it sits on the drawing, and the type says so honestly rather than defaulting it.

src/utils/grid.ts

```

1 export const GRID_COLS = 10; // A..J
2 export const GRID_ROWS = 6; // 1..6
3
4 /** Grid reference for a position, e.g. "C4". Derived, never stored. */
5 export function cellRef(planX: number, planY: number): string {
6   const col = Math.min(GRID_COLS - 1, Math.max(0, Math.floor(planX / (100 / GRID_COLS))));
7   const row = Math.min(GRID_ROWS - 1, Math.max(0, Math.floor(planY / (100 / GRID_ROWS))));
8   return `${String.fromCharCode(65 + col)}${row + 1}`;
9 }
10
11 /** Cell labels and line positions for the overlay, in percentages. */
12 export function gridCells(): { label: string; left: number; top: number }[] {
13   const cells: { label: string; left: number; top: number }[] = [];
14   const cw = 100 / GRID_COLS;
15   const ch = 100 / GRID_ROWS;
16   for (let r = 0; r < GRID_ROWS; r++) {
17     for (let c = 0; c < GRID_COLS; c++) {
18       cells.push({ label: `${String.fromCharCode(65 + c)}${r + 1}`, left: c * cw, top: r * ch });
19     }
20   }
21   return cells;
22 }

```

src/utils/planMap.ts

```

1 import warringtonSvg from '../plans/warrington-works.svg';
2 import wakefieldSvg from '../plans/wakefield-plant.svg';
3 import wiganSvg from '../plans/wigan-site.svg';
4
5 export const PLAN_BY_SITE: Record<string, string> = {
6   'Warrington Works': warringtonSvg,
7   'Wakefield Plant': wakefieldSvg,
8   'Wigan Site': wiganSvg,
9 };

```

This is the corrected version. The SVGs live in `src/plans/`, imported as modules, not in `public/plans/` referenced by string path. See the last bug log entry for why that distinction mattered.

src/hooks/useFloorPlanData.ts

```

1 import { useCallback, useEffect, useState } from 'react';
2 import type { Site, Asset, MaintenanceRequest } from '../types';
3
4 // Replace these three with whatever is actually exported from
5 // src/generated/services/ in your project. Open the files and read the
6 // real names, they follow the entity set (plural, capitalised), not the
7 // logical table name, and guessing wrong is the single most common
8 // first-run failure building this app.
9 import { Nvm_sitesService } from '../generated/services/Nvm_sitesService';

```

```

10 import { Nvm_assetsService } from '../generated/services/Nvm_assetsService';
11 import { Nvm_maintenancerequestsService } from '../generated/services/Nvm_maintenancerequestsService';
12
13 import type { Nvm_assetsBase } from '../generated/models/Nvm_assetsModel';
14
15 /** Dataverse suffix for human-readable labels on choices, lookups and currency. */
16 const FV = '@0Data.Community.Display.V1.FormattedValue';
17
18 /** Fallback if formatted values don't arrive alongside select. */
19 const STATUS_LABEL: Record<number, string> = {
20   1: 'New', 2: 'Triaged', 3: 'Assigned',
21   4: 'In Progress', 5: 'Complete', 6: 'Cancelled',
22 };
23
24 const OPEN = new Set(['New', 'Triaged', 'Assigned', 'In Progress']);
25
26 /**
27  * Dataverse's generated write type represents "no value" as undefined,
28  * never null, even though my own Asset type (and the UI built on it)
29  * uses null for "not yet placed". This helper is the one place that
30  * conversion happens, so nothing else in the file has to think about it.
31  */
32 function buildAssetPlacementPatch(
33   planX: number | null,
34   planY: number | null,
35 ): Partial<Omit<Nvm_assetsBase, 'nvm_assetid'>> {
36   return {
37     nvm_planx: planX ?? undefined,
38     nvm_plany: planY ?? undefined,
39   };
40 }
41
42 export function useFloorPlanData() {
43   const [sites, setSites] = useState<Site[]>([]);
44   const [assets, setAssets] = useState<Asset[]>([]);
45   const [requests, setRequests] = useState<MaintenanceRequest[]>([]);
46   const [loading, setLoading] = useState(true);
47   const [saving, setSaving] = useState(false);
48   const [error, setError] = useState<string | null>(null);
49
50   useEffect(() => {
51     let cancelled = false;
52
53     (async () => {
54       try {
55         const [s, a, r] = await Promise.all([
56           Nvm_sitesService.getAll({
57             select: ['nvm_siteid', 'nvm_sitename'],
58             orderBy: ['nvm_sitename asc'],
59           }),
60           // orderBy here must reference an Asset column. A Maintenance
61           // Request column (e.g. nvm_reportedon) throws a 400, see the
62           // bug log. This was the root cause of a full afternoon of
63           // "assets never load" with no visible error.
64           Nvm_assetsService.getAll({
65             select: [
66               'nvm_assetid', 'nvm_assetname', 'nvm_assettag',
67               'nvm_site_value', 'nvm_criticality',
68               'nvm_planx', 'nvm_plany',
69             ],
70             orderBy: ['nvm_assetname asc'],
71           }),
72           Nvm_maintenancerequestsService.getAll({
73             select: [
74               'nvm_maintenancerequestid', 'nvm_requestreference',
75               'nvm_asset_value', 'nvm_status', 'nvm_priority',
76               'nvm_reportedon', 'nvm_downtimehours',
77             ],
78             orderBy: ['nvm_reportedon desc'],
79           }),
80         ]);
81
82         if (cancelled) return;
83
84         const siteRows: Site[] = (s.data ?? []).map((x: any) => ({
85           id: x.nvm_siteid,
86           name: x.nvm_sitename,
87         }));
88
89         const reqRows: MaintenanceRequest[] = (r.data ?? []).map((x: any) => ({
90           id: x.nvm_maintenancerequestid,
91           reference: x.nvm_requestreference,
92           assetId: x.nvm_asset_value,
93           status: x[`nvm_status${FV}`] ?? STATUS_LABEL[x.nvm_status],
94           priority: x[`nvm_priority${FV}`],
95           reportedOn: x.nvm_reportedon,
96           downtime: x.nvm_downtimehours,
97         }));
98

```

```

39     const openByAsset = new Map<string, number>();
40     for (const req of reqRows) {
41         if (req.status && OPEN.has(req.status)) {
42             openByAsset.set(req.assetId, (openByAsset.get(req.assetId) ?? 0) + 1);
43         }
44     }
45
46     // Local state always uses null for "not placed", that's the
47     // Asset type's contract. Only the write path (below) converts to
48     // undefined, because that's what Dataverse's generated type wants.
49     const assetRows: Asset[] = (a.data ?? []).map((x: any) => ({
50         id: x.nvm_assetid,
51         name: x.nvm_assetname,
52         tag: x.nvm_assettag,
53         siteId: x.nvm_site_value,
54         criticality: x['nvm_criticality${FV}`'],
55         planX: x.nvm_planx ?? null,
56         planY: x.nvm_plany ?? null,
57         openCount: openByAsset.get(x.nvm_assetid) ?? 0,
58     }));
59
60     setSites(siteRows);
61     setAssets(assetRows);
62     setRequests(reqRows);
63 } catch (e: unknown) {
64     if (!cancelled) {
65         setError(e instanceof Error ? e.message : 'Failed to load plant data');
66     }
67 } finally {
68     if (!cancelled) setLoading(false);
69 }
70 })();
71
72 return () => { cancelled = true; };
73 }, []);
74
75 const placeAsset = useCallback(
76     async (assetId: string, planX: number, planY: number) => {
77         const x = Math.round(Math.min(100, Math.max(0, planX)) * 10) / 10;
78         const y = Math.round(Math.min(100, Math.max(0, planY)) * 10) / 10;
79
80         let previous: Asset | undefined;
81         setAssets(curr =>
82             curr.map(a => {
83                 if (a.id !== assetId) return a;
84                 previous = a;
85                 return { ...a, planX: x, planY: y };
86             }),
87         );
88
89         setSaving(true);
90         try {
91             await Nvm_assetsService.update(assetId, buildAssetPlacementPatch(x, y));
92             setError(null);
93         } catch (e: unknown) {
94             if (previous) {
95                 const snapshot = previous;
96                 setAssets(curr => curr.map(a => (a.id === assetId ? snapshot : a)));
97             }
98             setError(e instanceof Error ? e.message : 'Could not save the new position');
99         } finally {
100             setSaving(false);
101         }
102     },
103     [],
104 );
105
106 const clearAsset = useCallback(async (assetId: string) => {
107     let previous: Asset | undefined;
108     setAssets(curr =>
109         curr.map(a => {
110             if (a.id !== assetId) return a;
111             previous = a;
112             return { ...a, planX: null, planY: null }; // null, this is Asset state, not the Dataverse write
113         }),
114     );
115
116     setSaving(true);
117     try {
118         // NOT buildAssetPlacementPatch(null, null), that converts to
119         // undefined, which JSON.stringify drops from the request body
120         // entirely, sending an empty patch that clears nothing. Clearing
121         // needs an explicit null on the wire, which the generated type has
122         // no way to express, hence the deliberate double cast. See bug 11.
123         await Nvm_assetsService.update(
124             assetId,
125             { nvm_planx: null, nvm_plany: null } as unknown as Partial<Omit<Nvm_assetsBase, 'nvm_assetid'>>,
126         );
127         setError(null);

```

```

38     } catch (e: unknown) {
39       if (previous) {
40         const snapshot = previous;
41         setAssets(curr => curr.map(a => (a.id === assetId ? snapshot : a)));
42       }
43       setError(e instanceof Error ? e.message : 'Could not clear the position');
44     } finally {
45       setSaving(false);
46     }
47   }, []);
48
49   return { sites, assets, requests, loading, error, saving, placeAsset, clearAsset };
50 }

```

src/components/FloorPlan.tsx

```

1  import type { Asset } from '../types';
2  import { GRID_COLS, GRID_ROWS, gridCells } from '../utils/grid';
3
4  type Props = {
5    planUrl: string;
6    assets: Asset[];
7    selectedId: string | null;
8    onSelect: (id: string) => void;
9    editMode?: boolean;
10   onPlace?: (planX: number, planY: number) => void;
11   pendingAssetName?: string;
12 };
13
14 function colourFor(openCount: number) {
15   if (openCount === 0) return { fill: '#16A34A', ring: '#DCFCE7' };
16   if (openCount <= 2) return { fill: '#F59E0B', ring: '#FEF3C7' };
17   return { fill: '#DC2626', ring: '#FEE2E2' };
18 }
19
20 export function FloorPlan({
21   planUrl, assets, selectedId, onSelect,
22   editMode = false, onPlace, pendingAssetName,
23 }: Props) {
24   function handleClick(e: React.MouseEvent<HTMLDivElement>) {
25     if (!editMode || !onPlace) return;
26     const box = e.currentTarget.getBoundingClientRect();
27     const x = ((e.clientX - box.left) / box.width) * 100;
28     const y = ((e.clientY - box.top) / box.height) * 100;
29     onPlace(x, y);
30   }
31
32   const placed = assets.filter(a => a.planX !== null && a.planY !== null);
33
34   return (
35     <div
36       onClick={handleClick}
37       className="relative w-full overflow-hidden rounded-xl"
38       style={{ aspectRatio: '1000 / 600' }}
39     >
40       <img src={planUrl} alt="" className="block h-full w-full" />
41
42       {editMode && (
43         <svg
44           viewBox={`0 0 ${GRID_COLS * 100} ${GRID_ROWS * 100}`}
45           preserveAspectRatio="none"
46           className="pointer-events-none absolute inset-0 h-full w-full"
47         >
48           {Array.from({ length: GRID_COLS - 1 }, (_, i) => (
49             <line key={`v${i}`} x1={(i + 1) * 100} y1={0} x2={(i + 1) * 100} y2={GRID_ROWS * 100}
50               stroke="#6D28D9" strokeOpacity={0.18} strokeWidth={1.5} />
51           ))}
52           {Array.from({ length: GRID_ROWS - 1 }, (_, i) => (
53             <line key={`h${i}`} x1={0} y1={(i + 1) * 100} x2={GRID_COLS * 100} y2={(i + 1) * 100}
54               stroke="#6D28D9" strokeOpacity={0.18} strokeWidth={1.5} />
55           ))}
56           {gridCells().map(c => (
57             <text key={c.label} x={(c.left / 100) * GRID_COLS * 100 + 8} y={(c.top / 100) * GRID_ROWS * 100 + 20}
58               fontSize={16} fontFamily="system-ui, sans-serif" fill="#6D28D9" fillOpacity={0.3}>
59               {c.label}
60             </text>
61           ))}
62         </svg>
63       )}
64
65       {placed.map(a => {
66         const c = colourFor(a.openCount);
67         const on = a.id === selectedId;
68         return (
69           <button

```

```

70     key={a.id}
71     onClick={e => { e.stopPropagation(); onSelect(a.id); }}
72     aria-label={` ${a.name}, ${a.openCount} open requests`}
73     title={` ${a.name} open ${a.openCount}`}
74     className="absolute -translate-x-1/2 -translate-y-1/2 cursor-pointer rounded-full p-0 transition-all"
75     style={{
76       left: `${a.planX}%`,
77       top: `${a.planY}%`,
78       width: on ? 28 : 20,
79       height: on ? 28 : 20,
80       background: c.fill,
81       border: `3px solid ${on ? '#1F2937' : c.ring}`,
82     }}
83   />
84 );
85 }}}
86
87 {editMode && pendingAssetName && (
88   <div className="pointer-events-none absolute left-3 top-3 rounded-lg bg-violet-600 px-3 py-1.5 text-sm font-semibold text-white
89 shadow-lg">
90     Click to place: {pendingAssetName}
91   </div>
92 )}
93 </div>
94 );
95 }

```

`e.stopPropagation()` on the markers matters. Without it, clicking an existing marker in edit mode also registers as a click on the plan underneath, immediately repositioning whatever was pending.

I’ve not reproduced `App.tsx` here since my Tailwind conversion of it is already correct and working. See the shot list guide for the original structure it’s based on.

Bug log

Every bug I hit building this, in the order I hit them, each with the actual error, the root cause, and the fix. I’ve kept the pattern across most of them because it’s more useful than any single fix on its own.

1. power.config.json missing

Symptom: `pa app run` or `pa app push` couldn’t find the project config.

Cause: VS Code had reopened a parent folder rather than the project root, most likely via File then Open Recent selecting the wrong entry, or an unclean shutdown falling back to stale workspace state.

Fix: `cd` into the actual project root before running any `pa` command. I checked with `ls`, `package.json` and `power.config.json` should be sitting together.

Prevention: open the project by its own folder specifically (File then Open Folder, browsed directly), not via Recent. Get in the habit of `ls` before the first command in any new terminal.

2. Build output not found at ‘./dist’

Symptom: `pa app push` failed immediately.

Cause: push uploads compiled output, nothing had been built yet.

Fix: `npm run build` always before `pa app push`. Never implicit.

3. Generated service names didn’t match the imports

Symptom: TS2724, has no exported member named SitesService, did you mean Nvm_sitesService.

Cause: the generated services follow the entity set, plural, capitalised, prefixed, not a guessed generic name. SitesService was never real, it was always a placeholder meant to be replaced after checking the actual generated file.

Fix: I opened `src/generated/services/`, read the real exported names (Nvm_sitesService, Nvm_assetsService, Nvm_maintenancerequestsService), updated every import and every call site to match exactly.

This is the single most common first run failure building a code app against Dataverse. Never assume the name from the table’s logical name, always read it from the generated file.

4. Assets and Requests silently missing from the batch response

Symptom: Network tab showed a \$batch request returning 200 OK, but the multipart response body contained only one section, Sites, with no error and no sign the other two queries had ever been sent.

Cause: `orderBy: ['nvm_reportedon desc']` on the Assets query. `nvm_reportedon` is a Maintenance Request column, not an Asset column. Dataverse

rejected it with a 400 inside the batch, and because I'd changed the code to `Promise.allSettled` with a custom timeout wrapper, that rejection was caught and rethrown as one generic combined error rather than surfacing the actual OData error message.

Fix: I changed the Assets query's orderBy to `nvm_assetname asc`, a column that actually exists on that table.

Lesson: `Promise.allSettled` is a reasonable pattern, but it can mask the specific failing request behind a summarised error. When debugging nothing loads with no error, go straight to the Network tab's response body for the batch call. The real OData error is usually sitting right there, one section down.

5. Could not find a property named nvm_planx

Symptom: 400 Bad Request, could not find a property named nvm_planx on type Microsoft.Dynamics.CRM.nvm_asset.

Cause: the Plan X and Plan Y columns didn't exist on the Asset table yet. Not a naming mismatch, they genuinely hadn't been created.

Fix: I created the two columns in the maker portal (Whole Number type), then published the table.

6. Generated model still didn't have the columns after adding them

Symptom: TypeScript errors persisted even after the columns existed in Dataverse.

Cause: `pa app add data-source` generates model and service files as a one time snapshot. Adding columns to the table afterward doesn't retroactively update already generated files.

Fix:

```
pa app refresh data-source --name <data-source-name>
```

I reopened `Nvm_assetsModel.ts` fresh after refreshing. Never trust a previously viewed copy of a generated file once the schema has changed.

7. The null versus undefined boundary, hit twice

Symptom first time: TS2322, type null is not assignable to type number or undefined, on the Dataverse write call.

Symptom second time, after the first fix: TS2345, type object with planX undefined is not assignable to type Asset, types of property planX are incompatible, type undefined is not assignable to type number or null. This one only surfaced during `npm run build`, not during `pa app run`, because Vite's dev server doesn't run a full tsc type check the way a production build does.

Cause: two different type systems disagreeing on how to represent no value. The generated Dataverse model uses `number | undefined`. My own Asset type uses `number | null`. Fixing the mismatch in one direction (the write path) introduced the opposite mismatch in the other (local state), because I'd changed both to the same value instead of converting at the boundary.

Fix: null everywhere the value lives in my own Asset typed state, the fetch mapping, both optimistic local updates. Undefined only inside the literal object passed to `Nvm_assetsService.update(...)`, via the single `buildAssetPlacementPatch()` helper that does that one conversion in one place.

Lesson: when my own types and a third party generated type disagree on a null like value, I don't try to make them the same shape. I convert at the boundary, in one function, and never let the conversion leak into the rest of the codebase.

8. pa app run doesn't type check like a real build

Symptom: everything worked in Local Play. `npm run build` then failed with a type error that had apparently been there the whole time.

Cause: `pa app run` serves through Vite's dev server, which transpiles quickly and leniently, it does not run a full tsc type check first. `npm run build` runs `tsc -b && vite build`, and `tsc -b` refuses to proceed on any type error.

Lesson, the important one: a working `pa app run` session does not prove a working build. Before trusting it works, I run `npm run build` and treat a clean exit as the actual bar, not whether Local Play loads. This single fact explained most of a day's confusion about it works locally but breaks in prod, when the real story was it was never fully valid, one tool just didn't check.

9. Images 404 after publishing, despite working in Local Play

Symptom: RouteNotFound, the request URI ending in /plans/wakefield-plant.svg does not match any known API routes. Only in the published app. Local Play rendered the drawings perfectly the entire time.

First attempted fix, didn't work: referencing the SVGs from `public/plans/` via `import.meta.env.BASE_URL`. This is a standard Vite convention for static assets, but it assumes the app is served from a location that respects that base path the same way in every environment. A published code app routes static content through its own runtime proxy layer, which doesn't appear to honour public relative paths the same way a generic static host does.

Actual fix: I moved the SVGs into `src/plans/` and imported them as ES modules rather than referencing them as path strings. Vite bundles imported assets into `dist/assets/` with content hashed filenames, and resolves the correct URL for wherever the build is actually served, at build time, not at runtime. I confirmed this in the published app's Network tab, a request to the runtime proxy path ending in assets/wakefield-plant-D5x7KVBO.svg returning 200 OK.

Lesson: for a code app specifically, don't reference public assets by absolute or BASE_URL relative path if there's any chance the app will be embedded or proxied differently once published. Import static assets as modules. It costs one line per asset and removes an entire class of environment specific path bugs.

10. Diagnosing false leads from a static test

Symptom: I ran `npm run build` then `npx serve dist` to test the production build in isolation, and the app appeared stuck on Loading plant data forever.

What this actually confirmed, correctly: the build was structurally sound, React mounted, routing worked, the shell rendered. It did not and could never test Dataverse connectivity, because a bare static server has no Power Platform authentication context. The infinite loading state was the correct outcome for that specific test, not a new bug.

Lesson: `npx serve dist` is a genuinely useful test for is the compiled output broken, but it can't validate anything that depends on the Power Apps host context, auth, Dataverse calls, or, as it turned out, the runtime proxied asset routing that only exists once actually published. I need to know what each test environment can and can't tell me before drawing a conclusion from it.

Environment	Type-checks?	Dataverse works?	Real asset routing?
pa app run (Local Play)	No	Yes	No, dev server, not the proxy
npx serve dist	Not applicable	No, no auth context	No, plain static server
Published app	Not applicable	Yes	Yes, the only one that's real

Nothing short of the actual published app can confirm the actual published behaviour. Local Play and a static serve are both useful for narrowing down which layer a problem is in, but neither can stand in for the real thing.

11. Clear worked visually, then reverted on refresh

Symptom: clicking Clear on a placed asset removed its marker immediately. Refreshing the page, or leaving and returning to edit mode, brought it back at its old position, as if nothing had been sent.

First hypothesis, ruled out: that the write was failing and being rolled back. It wasn't, no error appeared, saving completed normally, and the Network tab showed the PATCH request returning 200 OK.

Actual cause: the request body was empty. `clearAsset` was reusing the same `buildAssetPlacementPatch()` helper as `placeAsset`, which converts null to undefined before sending, correct for satisfying the generated type, but undefined object properties are dropped entirely by `JSON.stringify`. The PATCH request Dataverse received had genuinely no fields in it, so there was nothing to clear. The 200 was real, it was just a 200 for an empty patch.

```
1 // Sends {} - undefined keys don't survive JSON.stringify
2 await Nvm_assetsService.update(assetId, buildAssetPlacementPatch(null, null));
```

Fix: undefined and null are not interchangeable here even though the generated type only has a slot for one of them. Undefined means field not present in this patch, leave it alone. Null means set this field to nothing. Clearing needs the second one, explicitly, on the wire, and the generated type has no way to express that intent at all. The fix is a deliberate, documented cast past the generated type for this one call, not a change to the type itself:

```
1 await Nvm_assetsService.update(
2   assetId,
3   { nvm_planx: null, nvm_plany: null } as unknown as Partial<Omit<Nvm_assetsBase, 'nvm_assetid'>>,
4 );
```

The double cast (as unknown as) is intentional. TypeScript won't allow the direct cast because the shapes genuinely don't match, and that mismatch is the whole point being documented, not an error to silence quietly. A single `as any` would work too, the explicit target type is only there so a future reader sees exactly what shape is actually being sent.

Lesson: undefined and null were already known to need converting at the boundary between my own types and Dataverse's generated ones, that's bug 7. This is the sharper version of the same lesson. Sometimes the generated type doesn't just require a different value, it's structurally unable to express the value I need at all. When that happens, the fix isn't finding the correct typed value, there isn't one, it's a narrow, commented cast at the exact point where intent and type diverge, with the `JSON.stringify` behaviour of undefined written down so I don't rediscover this by accident six months later.

Summary: the five root causes behind everything

Eleven bugs, but really five categories, and they're worth remembering for the next code app rather than just this one.

Generated names are never what I'd guess. Read them from the file, every time, especially after a schema change that requires `pa app refresh data-source`.

My types and Dataverse's generated types will disagree on edge values like null and undefined. Convert at one boundary function. Never let the two shapes merge.

Undefined isn't a safer, more compatible version of null, it's a different instruction, and `JSON.stringify` silently drops it. No value (omit the field) and clear this field (send null) look similar in TypeScript and are not the same request on the wire. When a generated type can't express the one I actually need, I cast past it deliberately and write down why in a comment, rather than reaching for whichever value happens to satisfy the compiler.

`pa app run` is not a build. It doesn't type check the way `npm run build` does. I treat a clean `npm run build` as the actual bar for this works, not a working dev session.

Static assets under public don't survive a published code app's routing the same way they do in local dev. Import them as modules, let the bundler

resolve the path.